

Project 1

Deliverables

Part 1: Connect your workspace to GitHub

Link for the repo: <https://github.com/josuefernandezcanela-tamu/istm637-lakehouse-josuefernandezcanela>

The screenshot shows the GitHub repository page for 'istm637-lakehouse-josuefernandezcanela'. The repository is public and has 7 commits. The commit history shows three recent commits: 'Fix VOL_PATH verification cell after debugging' (4 days ago), 'Add files via upload' (18 hours ago), and 'Added my NetID to the README' (4 days ago). The README file is visible, showing the repository name and the user's NetID. The right sidebar contains links to Readme, Activity, Stars, Watching, and Forks, as well as sections for Releases, Packages, and Contributors.

istm637-lakehouse-josuefernandezcanela

1 Branch 0 Tags

Go to file

Add file

Code

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 1

istm637-lakehouse-josuefernandezcanela

main

Open in editor

Send feedback

Share

Create

Search

Type

Owner

Last modified

Name	Type	Owner	Created at	Last updated at
.git	Folder	josuefernandezc...	Jul 26, 2026, 03:3...	Jul 26, 2026, 03:39 PM
ISTM637_Databricks_Project_Starter	Notebook	josuefernandezc...	Jul 26, 2026, 03:3...	Jul 26, 2026, 03:44 PM
README.md	File	josuefernandezc...	Jul 26, 2026, 03:3...	-

GitHub Dialogue:

istm637-lakehouse-josuefernandezcanela

Git CLI enabled

Send feedback

Branch: main

Create Branch

99015e8

Web Terminal

Pull

Changes

Settings

1 changed file

README.md

Did a slight change on one line

Description (optional)

Commit & Push

README.md

@@ -6,8 +6,7 @@

Summary

-An end-to-end governed data + AI product built on Databricks Free Edition, starting from three CSV files of oil & gas production data

-schema in Unity Catalog, adds AI-assisted metadata, exposes the data through a Genie Space and an AI/BI dashboard, trains and registers

+An end-to-end governed data + AI product built on Databricks Free Edition, starting from three CSV files of oil & gas production data

Architecture

istm637-lakehouse-josuefernandezcanela

Public

Pin

Watch 0

Fork 0

Star 0

main

1 Branch

0 Tags

Go to file

Add file

Code

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 1

josuefernandezcanela-tamu

Languages

istm637 — Building a Governed Lakehouse

Author: Josue Fernandez Canela

NetID: jfernandezcanela

Catalog: istm637_jfernandezcanela

Summary

An end-to-end governed data + AI product built on Databricks Free Edition, starting from three CSV files of oil & gas production data. The project stands up a star schema in Unity Catalog, adds AI-assisted metadata, exposes the data through a Genie Space and an AI/BI dashboard, trains and registers a production forecasting model, and serves history + forecast through a lightweight Databricks App.

Architecture

Part 2: Ingest the tables into Unity Catalog with Lakeflow

New Pipeline 20... Send feedback Run pipeline Edit pipeline Settings Schedule Share

Run: Jul 26, 2026, 04:25 PM - Completed Select tables for refresh

Pipeline details

- Pipeline ID: 0b648a0d-3353-444e-98ca-055761efb6...
- Pipeline type: ETL pipeline
- Creator: josuefernandezcanela@tamu.edu
- Owner: josuefernandezcanela@tamu.edu
- Run as: josuefernandezcanela@tamu.edu
- Source code: 1 folder [Open in Editor](#)

Tables Performance 3

Name	Duration	Output r...	Expectations	Incrementalization
dim_date	4s	547	-	Full recompute
dim_well	4s	50	-	Full recompute
fact_production	5s	23K	2 met	Full recompute

Catalog Serverless Starter Warehouse Serverless 2XS

oilgas Use with BI tools Share Create

Overview Details Permissions Policies

Description Oil & gas star schema

Filter tables

Tables Volumes 1 Models 0 Functions 0 Secrets 0 Services 0 Connections 0

Name	Owner	Created at	Popularity	Comment
dim_date	josuefernandezcanela@t...	Jul 26, 2026, 04:27 PM	*****	Calendar date dimensio...
dim_well	josuefernandezcanela@t...	Jul 26, 2026, 04:27 PM	*****	Well master data: one ro...
fact_production	josuefernandezcanela@t...	Jul 26, 2026, 04:27 PM	*****	Daily production fact ta...

About this schema

Owner: josuefernandezcanela@tamu.edu

Tags

Add tags

Policies

New policy

Just now (5s)

8

Python

```
# Confirm the Lakeflow pipeline created the three tables in Unity Catalog
for t in ["dim_well", "dim_date", "fact_production"]:
    try:
        n = spark.table(f"{CATALOG}.{SCHEMA}.{t}").count()
        print(f"{t:18s} -> {n:>7,} rows ✓")
    except Exception as e:
        print(f"{t:18s} -> NOT FOUND yet - run the Lakeflow pipeline first")
# Expected: ~50 wells, ~547 dates, ~22,800 production rows
```

> [See performance \(3\)](#)

✓ Checked for improvements

dim_well	->	50 rows	✓
dim_date	->	547 rows	✓
fact_production	->	22,806 rows	✓

Part 3: Add metadata with the AI assistant

This is the production_facts table with the comments from the Genie.

Edit column comments

Column	Comment
production_id int	Primary Key. Unique identifier for each production record, enabling easy tracking and reference of specific entries in the dataset.
well_id string	Identifier for the oil or gas well from which the production data is sourced, allowing for the differentiation between multiple wells in the dataset.
date_id int	Represents the specific date associated with the production data, facilitating time-based analysis of production trends.
oil_bbl double	The quantity of oil produced, measured in barrels, providing insight into the well's production capacity and performance.
gas_mcf double	The volume of natural gas produced, measured in thousand cubic feet, which is essential for evaluating overall production metrics.
water_bbl double	The amount of water produced alongside oil and gas, measured in barrels, which is critical for understanding production efficiency and environmental impact.
downtime_hours double	The total number of hours the well was not operational due to various reasons, important for assessing well productivity and efficiency.
choke_size_64ths int	Indicates the size of the choke opening, measured in 64ths of an inch, which affects the flow rate of oil and gas extraction.
tubing_pressure_psi int	The pressure within the tubing, measured in pounds per square inch, which is vital for monitoring well performance and ensuring safety.
casing_pressure_psi int	The pressure within the casing, measured in pounds per square inch, used to evaluate the integrity and stability of the well structure.
_rescued_data string	Contains any data that was salvaged or recovered during processing, useful for understanding potential discrepancies or gaps in the dataset.

This is after the edits:

Edit column comments

Column	Comment
production_id int	Primary Key. Unique identifier for each production record, enabling easy tracking and reference of specific entries in the dataset.
well_id string	Foreign key → dim_well.well_id
date_id int	Foreign key → dim_date.date_id
oil_bbl double	Oil produced that day in barrels.
gas_mcf double	Gas produced that day measured on thousand cubic feet.
water_bbl double	Amount of water produced that day, measured in barrels, which is critical for understanding the well's output composition.
downtime_hours double	Total hours the well was down (0–24), valuable for maintenance and operational efficiency analysis.
choke_size_64ths int	Surface choke setting in 64ths of an inch
tubing_pressure_psi int	Flowing tubing pressure (psi)
casing_pressure_psi int	Pressure within the casing of the well (psi), is essential for monitoring well integrity and safety.
_rescued_data string	Information that has been recovered or adjusted from raw input, useful for maintaining data quality and consistency.

0 comments unsaved

✓ Save all

Dim_well table before the edits:

Column	Comment
well_id string	A unique identifier assigned to each well in the dataset, facilitating precise referencing and data retrieval.
well_name string	The official name of the well, which can provide insight into its identity and purpose.
operator string	The company or organization responsible for the operation and management of the well.
field_name string	The name of the field where the well is located, helping to categorize the production area.
basin string	The geological basin name indicates the larger geographic area where the well is situated.
state string	The U.S. state in which the well is located, useful for regulatory and reporting purposes.
county string	The county within the state where the well is located, providing a more localized reference.
target_formation string	The specific geological formation that the well is intended to tap into for resource extraction.
well_type string	The classification of the well, such as oil, gas, or injection, reflecting its intended use.
lateral_length_ft int	The horizontal length of the well bore measured in feet, which can affect production potential.
latitude double	The geographic coordinate representing the north-south position of the well on the Earth's surface.
longitude double	The geographic coordinate representing the east-west position of the well on the Earth's surface.
spud_date date	The date when drilling commenced, marking the beginning of the well's development.
completion_date date	The date when the well was completed and officially became operational.
status string	The current operational status of the well, which may indicate if it is active, inactive, or abandoned.
initial_oil_potential_bo... int	The initial production potential of the well, expressed in barrels of oil per day, indicating expected performance.
_rescued_data string	Any additional data that has been recovered or salvaged during processing, which may contain useful information.

After the edits:

Edit column comments

Column	Comment
well_id string	Primary Key. A unique identifier assigned to each well in the dataset, e.g. WELL-0007
well_name string	Human readable name of the well.
operator string	The company responsible for the operation of the well.
field_name string	Field the well belongs to.
basin string	The geological basin (Permian, Delaware, Eagle Ford, Bakken, DJ).
state string	US state code (TX, NM, ND, CO)
county string	The county within the state where the well is located,.
target_formation string	Producing formation (e.g. Wolfcamp A, Bone Spring)
well_type string	Wellbore trajectory type: Horizontal, Directional, or Vertical.
lateral_length_ft int	The horizontal length of the well bore measured in feet, 0 for vertical wells.
latitude double	The geographic coordinate representing the north-south position of the well on the Earth's surface (decimal degrees).
longitude double	The geographic coordinate representing the east-west position of the well on the Earth's surface (decimal degrees).
spud_date date	The date when drilling started, marking the beginning of the well's development.
completion_date date	The date when the well was completed.
status string	Current operational status: Producing or Shut-in.
initial_oil_potential_bo... int	The initial production potential of the well, expressed in barrels of oil per day, indicating expected performance.
_rescued_data string	Any additional data that has been recovered or salvaged during processing, which may contain useful information.

3 comments unsaved

✓ Save all

AI suggestions were mostly correct; however, I found them too wordy. I summarized them and edited them to be closer to the data dictionary. The AI also didn't identify the primary and foreign keys and made a few mistakes with the measuring units or had assumptions on how variables like quarter_name were named. In summary, I had to remove the wordiness, indicate the primary and foreign keys, updated measuring units and made everything closer to the data dictionary.

Tags:

Definition SQL

```
1 CREATE MATERIALIZED VIEW 'istm637_jfernandezcanela'.oilgas.`dim_date` (  
2   date_id INT COMMENT 'Primary key. Date represented as a yyyyymmdd integer.',  
3   calendar_date DATE COMMENT 'Holds the actual calendar date.',  
4   year INT COMMENT 'Indicates the calendar year.',  
5   quarter INT  
... 32 more lines
```

Popularity

Current pipeline update status

Completed Jul 26, 2026, 04:27 PM (7 d ago)

See refresh details

Refresh schedule

No refresh schedules

Quality

Not enabled

Data Quality Monitoring is not enabled for this table

Enable

Policies

New policy

Tags

domain:calendar layer:silver pii:None

Row filter

Add filter

Insights

Top users

josuefernandezcanela@tamu.edu

Filter columns...

Column	Type	Popularity	Comment	Tags	Column masking
date_id	int		Primary key. Date represented as a yyyyymmdd integer.		
calendar_date	date		Holds the actual calendar date.		
year	int		Indicates the calendar year.		
quarter	int		Numerical representation of the quarter, where values range from 1 to 4, facilitating quarter-based reporting.		
quarter_name	string		String representation of the quarter name, e.g. Q1 2024.		

Part 4: Build a Genie Space

Question	Result	Notes
Which basin produced the most oil overall?	Correct	Permian 2.2M bbl vs Eagle Ford 2.1M, matches my validated trusted query exactly (2,199,769 vs 2,148,451).
Show monthly oil production trend for 2024	Correct	Peak in May (731K), decline to Dec (429K), consistent with trusted query and the "hyperbolic decline" behavior described in the project overview.
Average water cut for Permian wells	Correct	52.2% — reasonable basin-wide average (well-level leaderboard showed individual outlier wells up to 70%+, so a basin average in the low 50s is consistent).
Which operator has the most producing wells?	Correct	Coterra & Marathon Oil tied at 7, correctly filtered on status = 'Producing', which is exactly the term I clarified in my instructions
Top 5 wells by total gas production	Correct	Weld 35-34 leads at 1.13M mcf , nice touch, Genie even added GOR to the table on its own, showing it picked up my key-terms definition.

No changes were made. My guess is that I provided enough information in the instructions.

General Instructions

Add general instructions on how you want this Genie Agent to behave.

Domain: oil & gas daily production
Grain: one row per well per producing day

Key terms:

- Production means oil_bbl unless otherwise stated
- GOR (gas-oil ratio) = gas_mcf / oil_bbl
- Water cut = water_bbl / (oil_bbl + water_bbl)
- well_type = drilling trajectory (Horizontal/Directional/Vertical); status = current operations (Producing/Shut-in) – do not conflate these
- basin = broad geological region (e.g. Permian); field_name = more specific producing area within a basin

Units:

- Oil: barrels (bbl)
- Gas: thousand cubic feet (mcf)
- Pressure: psi
- choke_size_64ths: 64ths of an inch
- downtime_hours: 0–24 per day (hours the well was non-operational)

Joins:

- fact_production.well_id → dim_well.well_id
- fact_production.date_id → dim_date.date_id

Question: How many horizontal wells are currently producing.

How many horizontal wells are currently producing?

Thinking complete

I'll query the well data to find the count of horizontal wells that are currently producing.

Executed query

There are 21 horizontal wells currently producing. 1

Sources (1)

Is this useful?

Share Download PDF

What's the best well?

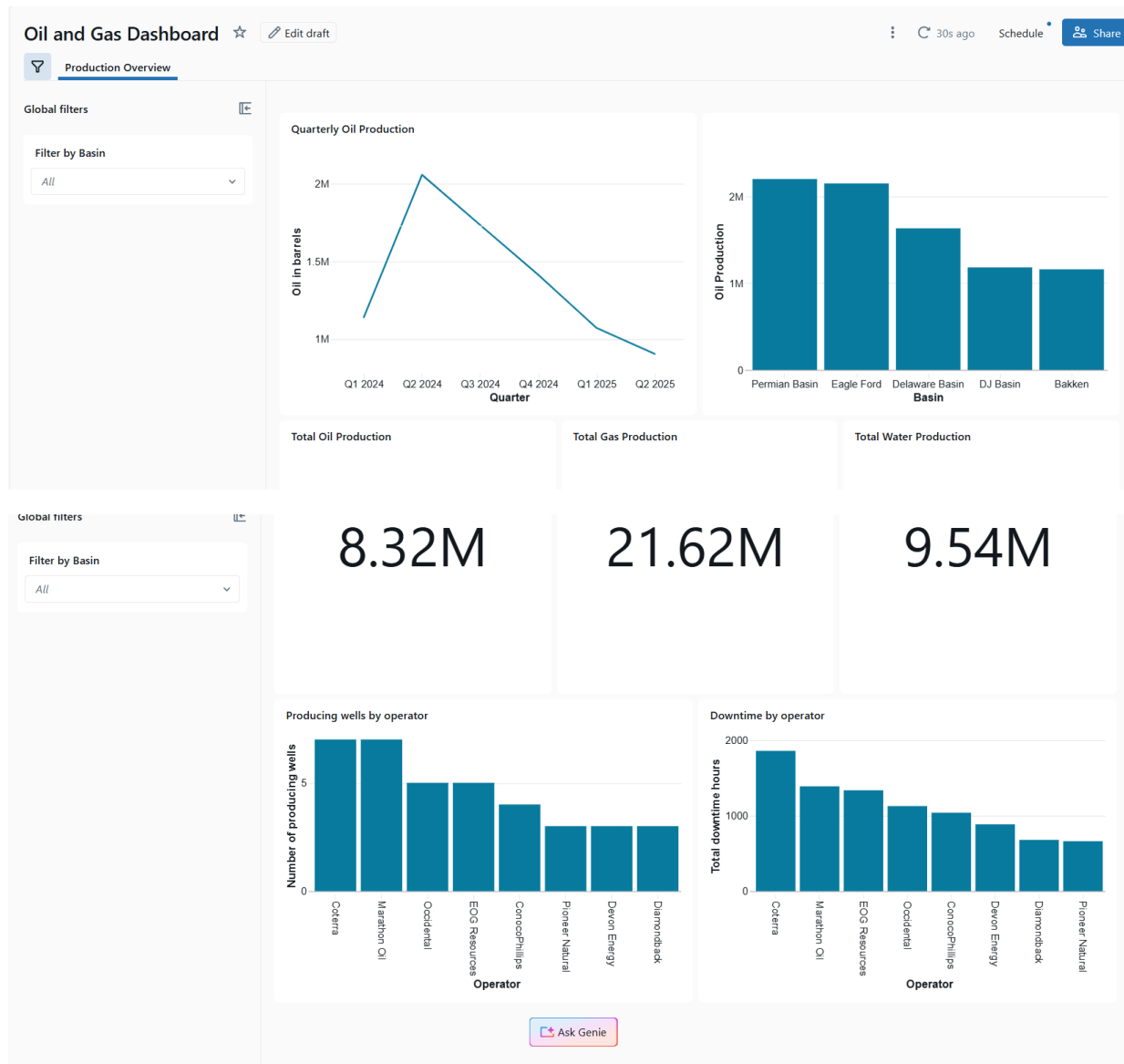
Count of horizontal producing wells

	horizontal_producing_wells
1	21

Hide code

```
SELECT
  COUNT(*) AS horizontal_producing_wells
FROM
  `istm637_jfernandezcanela`.`oilgas`.`dim_well`
WHERE
  `well_type` = 'Horizontal'
  AND `status` = 'Producing'
```

Part 5: Build an AI/BI Dashboard



The dashboard was built using the AI/BI Dashboard's visual Data Model rather than a single hand-written SQL query. I added `fact_production`, `dim_well`, and `dim_date` as datasets and explicitly defined the join relationships (`fact_production.well_id` → `dim_well.well_id` and `fact_production.date_id` → `dim_date.date_id`, both many-to-one). Each visualization queries this relationship graph directly through field selections in the chart editor, so Databricks generates the underlying SQL per chart rather than my writing one consolidated query. This approach kept the joins declarative and reusable across all five visualizations and the page-wide basin filter, rather than repeating the same joins in five separate hand-written queries.

Model

Catalog

Serverless Starter Warehouse Serverless 2XS

Catalog

Type to search

My organization

workspace

system

istm637_jfernandezcanela

default

information_schema

oilgas

Tables (4)

dim_date

dim_well

fact_production

well_forecast

Volumes (1)

raw

Functions (1)

oil_rate_predictor

Models (1)

oil_rate_predictor

Shares received

samples

Catalog Explorer > istm637_jfernandezcanela > oilgas >

oil_rate_predictor

Share Serve this model

Overview Details Permissions Policies Lineage

Description

AI generate Add

Versions

Status	Version	Tags	Aliases	Deployment jo...	Active endpoints	Comment
	Version 1		@ champion			

Activity log

Activity logs capture updates from deployment jobs. Learn more.

Time	Version	Log
2026-07-30 01:03:10	Version 1	Model version 1 registered by user josuefernandezcanela@tamue.edu.

About this model

Owner josuefernandezcanela@tamue.edu

Deployment job

To manage the model lifecycle, link a deployment job. A deployment job can automatically trigger steps such as evaluation, approval, and deployment each time a new model version is created. Learn more.

Connect deployment job

Tags

Add tags

bricks.com/explore/data?o=7474658965502079

Part 6: Train a prediction model & register it in Unity Catalog

▶ ✓ 07:10 PM (<1s) 9

```
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

pred = model.predict(X_test)
mae = mean_absolute_error(y_test, pred)
rmse = mean_squared_error(y_test, pred) ** 0.5
r2 = r2_score(y_test, pred)
print(f"MAE = {mae:.1f} bbl/day")
print(f"RMSE = {rmse:.1f} bbl/day")
print(f"R2 = {r2:.3f}")
# TODO (extend): try adding interaction features or a different regressor and beat this R2.
```

MAE = 33.5 bbl/day
RMSE = 70.0 bbl/day
R2 = 0.933

Try machine learning in Genie Code: Genie Code can help you build, debug, and iterate on machine learning code. [Open Genie Code](#) to get started.
[Don't show me this again](#) ?

display(spark.createDataFrame(fc))

> [See performance \(3\)](#)

✓ Checked for improvements

> fc: pandas.core.frame.DataFrame = [day_ahead: int64, predicted_oil_bbl: float64]

180-day forecast for WELL-0001: total = 51,855 bbl

Table

	1.2 day_ahead	1.2 predicted_oil_bbl
1	1	295.8932940508315
2	2	295.8932940508315
3	3	295.8932940508315
4	4	295.78097653894025
5	5	295.78097653894025
6	6	295.78097653894025
7	7	295.1305140695178
8	8	295.1305140695178
9	9	295.1305140695178
10	10	295.1305140695178
11	11	295.1305140695178
12	12	293.40228504660666
13	13	293.40228504660666
14	14	293.40228504660666
15	15	293.40228504660666

180 rows | 6.666s runtime

Refreshed Now

Catalog

Serverless Starter Warehouse Serverless ZXS

Catalog

Type to search

My organization

workspace

system

istm637_jfernandezcanela

default

information_schema

oilgas

Tables (4)

dim_date

dim_well

fact_production

well_forecast

Volumes (1)

raw

Functions (1)

oil_rate_predictor

Models (1)

oil_rate_predictor

Shares received

samples

Catalog Explorer > istm637_jfernandezcanela > oilgas >

oil_rate_predictor

Share Serve this model

Overview Details Permissions Policies Lineage

Description

AI generate Add

Versions

Status	Version	Tags	Aliases	Deployment jobs	Active endpoints	Comment
✓	Version 1		@ champion			

Activity log

Activity logs capture updates from deployment jobs. [Learn more.](#)

Time	Version	Log
2026-07-30 01:03:10	Version 1	Model version 1 registered by user josuefernandezcanela@tamu.edu.

About this model

Owner josuefernandezcanela@tamu.edu

Deployment job

To manage the model lifecycle, link a deployment job. A deployment job can automatically trigger steps such as evaluation, approval, and deployment each time a new model version is created. [Learn more.](#)

Connect deployment job

Tags

Add tags

Part 7: Build & deploy a simple data app

Prompt used to create the app:

Create a small Streamlit Databricks App in this workspace folder:
/Workspace/Users/josuefernandezcanela@tamu.edu/oil-well-production-app

The app must be a simple, lightweight, one-page oil-well production application.

Use these fully qualified Unity Catalog tables:

istm637_jfernandezcanela.oilgas.dim_well
istm637_jfernandezcanela.oilgas.dim_date
istm637_jfernandezcanela.oilgas.fact_production
istm637_jfernandezcanela.oilgas.well_forecast

Requirements:

1. Generate these three files in the folder:
 - app.py
 - app.yaml
 - requirements.txt
2. Use Streamlit for the user interface.
3. Use Databricks app service-principal authentication, the Databricks SQL Connector for Python, and a SQL warehouse resource. Do not use a personal access token or hard-code credentials.
4. Add one dropdown for selecting a well. Populate it from dim_well. Use well_id as the value. If a well_name column exists, display the name and ID; otherwise display well_id.
5. For the selected well, show two line charts:
 - Chart 1: Monthly historical oil production.
Join fact_production to dim_date and aggregate oil_bbl by calendar month inside SQL so only a small result set is returned.
 - Chart 2: Forecast production.
Read day_ahead and predicted_oil_bbl from well_forecast for the selected well, ordered by day_ahead.
6. Use parameterized SQL queries for the selected well. Do not build SQL by concatenating user input.
7. Do not load the MLflow model in the app. Use the pre-computed well_forecast table.
8. Add helpful titles, axis labels, loading indicators, empty-result warnings, and basic exception handling.
9. Cache the well-list query where appropriate.
10. Configure app.yaml to launch Streamlit and obtain the SQL warehouse ID from a Databricks App SQL warehouse resource.
11. requirements.txt should contain only the packages needed for this app, including:
 - streamlit
 - pandas
 - databricks-sdk
 - databricks-sql-connector

After generating the files, explain any table or column assumptions you made.

Then I did a correction:

Two corrections: fact_production does not have a production_date column — it has date_id (int), which is a foreign key to dim_date.date_id. dim_date does not have date_key or year_month — it has date_id (int, the join key), calendar_date (date), year (int), month (int), and month_name (string). Please update the monthly history query to join fact_production.date_id = dim_date.date_id, and aggregate using dim_date.year and dim_date.month instead of production_date/year_month.

Thoughts: The user is providing corrections about the table schema: >

I'll update the app.py file to fix the historical query with the correct column names and join logic.

2 steps

Updated! I've corrected the historical query in app.py:

Changes made:

- Join condition: `f.date_id = d.date_id` (instead of `f.production_date = d.date_key`)
- Grouping: `GROUP BY d.year, d.month` (instead of `d.year_month`)
- Ordering: `ORDER BY d.year, d.month` (instead of `d.year_month`)
- Month display: Created `YYYY-MM` format using `CONCAT(CAST(d.year AS STRING), '-', LPAD(CAST(d.month AS STRING), 2, '0'))`

The query now properly joins on the integer foreign key relationship and aggregates by year and month columns.

App deployed:

The screenshot displays the AWS App Studio console for an application named 'oil-well-production-app'. The interface includes a left-hand navigation menu with options like Overview, Authorization, Deployments, Logs, Environment, Insights, and Settings. The main content area shows the app's status as 'Running' with a URL. Below this, the 'Deployment history' section lists several successful deployment events from August 2, 2026, at 07:27:19 PM EDT. On the right, the 'About the App' section provides details about the creator (Josuefernandezcanela@tamuj.edu), creation time (3 days ago), and app ID (d310f4b0-1b21-431e-8e96-5fd529f4_41bf). The 'Description' section states that the app displays production history and forecast for a selected well from the ISTM 637 oil & gas star schema. The 'Compute' section indicates a medium instance type with up to 2 vCPUs, 6 GB memory, and 0.5 DBU/hour. A 'Tags' section with an 'Add tags' button is also visible.



App Not Available

Sorry, the Databricks app you are trying to access is currently unavailable. Please try again later.

Since the app UI can't show history and forecast for a chosen well, I ran SQL queries:

Run all (1000)

2 minutes ago (17s)

workspace.default

Generate

1

2

3

4

5

SELECT d.calendar_date, f.oil_bbl

FROM istm637_jfernandezcanela.oilgas.fact_production f

JOIN istm637_jfernandezcanela.oilgas.dim_date d ON f.date_id = d.date_id

WHERE f.well_id = 'WELL-0001'

ORDER BY d.calendar_date;

Add parameter

Table

+

	calendar_date	1.2 oil_bbl
1	2024-05-13	1188
2	2024-05-14	1227.3
3	2024-05-15	1178.8
4	2024-05-16	1227.9
5	2024-05-17	1141
6	2024-05-18	1313
7	2024-05-19	1188.3
8	2024-05-20	1143.2
9	2024-05-21	1205.1
10	2024-05-22	1285.5
11	2024-05-23	1207.6
12	2024-05-24	1180.3
13	2024-05-25	1128.2
14	2024-05-26	1098.6
15	2024-05-27	1151.4
16	2024-05-28	1065.7
17	2024-05-29	1145.4
18	2024-05-30	1030.8

414 rows

17.368s runtime

[See performance \(1\)](#)

✓

Checked for improvements

Refreshed 2m ago

▶ Run all (1000) ✓ Just now (1s) workspace.default ☆

1 SELECT day_ahead, predicted_oil_bbl

2 FROM istm637_jfernandezcanela.oilgas.well_forecast

3 WHERE well_id = 'WELL-0001'

4 ORDER BY day_ahead;

Add parameter

Table +

Q 🔍 📄 📊 📁 ⌵ ✕

	1.2 day_ahead	1.2 predicted_oil_bbl
1	1	295.8932940508315
2	2	295.8932940508315
3	3	295.8932940508315
4	4	295.78097653894025
5	5	295.78097653894025
6	6	295.78097653894025
7	7	295.1305140695178
8	8	295.1305140695178
9	9	295.1305140695178
10	10	295.1305140695178
11	11	295.1305140695178
12	12	293.40228504660666
13	13	293.40228504660666
14	14	293.40228504660666
15	15	293.40228504660666
16	16	293.40228504660666
17	17	293.40228504660666
18	18	293.40228504660666

⌵

180 rows | 1.218s runtime [See performance \(1\)](#) ✓ Checked for improvements Refreshed Now ⚙

▶ Run all (1000) ✓ Just now (1s) workspace.default ☆

1 GRANT USE SCHEMA ON SCHEMA istm637_jfernandezcanela.oilgas TO `d310f4b0-1b21-431e-8e96-5fdc529f41bf`;

2 GRANT SELECT ON SCHEMA istm637_jfernandezcanela.oilgas TO `d310f4b0-1b21-431e-8e96-5fdc529f41bf`;

Add parameter

← Results 2 of 2 →

Table +

📄 ⌵ ✕

OK

▶ Run all (1000) ✓ Just now (<1s) workspace.default ☆

⋮ Edit 🟢 📅 📄 💬 📶

1 | SHOW GRANTS ON SCHEMA `istm637_jfernandezcanela.oilgas;`

Add parameter

Table + 🔍 🏠 📄 🗨 ⤴ ⤵

	^A _C Principal	^A _C ActionType	^A _C ObjectType	^A _C ObjectKey
1	d310f4b0-1b21-431e-8e96-5fdc529f41...	SELECT	SCHEMA	istm637_jfernandezcanela.oilg...
2	d310f4b0-1b21-431e-8e96-5fdc529f41...	USE SCHEMA	SCHEMA	istm637_jfernandezcanela.oilg...

Part 8: OpenSharing with a classmate

Delta Sharing on Databricks Free Edition cannot be completed end-to-end. External Delta Sharing is disabled at the metastore level and the setting is not available to Free Edition users, so the exchange is blocked at recipient creation rather than at verification.

The screenshots below show the full setup as far as the platform permits, with my classmate Alexis Cole:

Catalog Explorer > Shares >

Share data

1 Create share

2 Add data assets

3 Add notebooks

4 Add recipients

⚠ External sharing is currently disabled in metastore configuration. To share with external recipients, please enable Delta Sharing. Enable External OpenSharing

Share name*

aws:us-east-2:7a165a6f-9422-448d-80bd-fb7

Description

Sharing for Data Warehouse project.

35 / 255

Cancel

Save and continue

Share data

- 1 Create share
- 2 **Add data assets**
- 3 Add notebooks
- 4 Add recipients

Select assets across catalogs and schemas

Type to search...

For you All

- istm637_jfernandezcanela
 - default
 - oilgas
 - Tables (4)
 - ☐ dim_date
 - ☒ dim_well
 - ☐ fact_production
 - ☐ well_forecast
 - Volumes (1)
 - workspace

Type	Name	Alias	History	Partition
	istm637_jfernandezcanela...		-	-

Tables are shared with full history and commit log by default. You can choose to exclude sharing history for one or more tables.

Cancel

Back

Save and continue

< Create a new recipient



Recipient name

AlexisCole

Recipient type

Databricks



The recipient is a Databricks user and intends to use OpenSharing on the Databricks platform

Open 



The recipient is not a Databricks user or intends to use OpenSharing on an external platform

Sharing identifier (Databricks recipient only)*

aws:us-east-2:7a165a6f-9422-448d-80bd-fb7e060fd5fc

Enter the recipient's Databricks sharing identifier. If the recipient is not a Databricks user, ask your account admin to enable external sharing to create external recipients



Your current org name will appear as 'aws:us-east-2:1578150a-43ed-48f8-bb3f-66d2b23746a8' on the recipient/provider side, please consider updating it [here](#).

Description



External Delta Sharing is not enabled on the metastore. Please contact the metastore admin to enable External Delta Sharing.

Back

Create and add recipient

